Report

Setayesh Saeidi - 40131851

Note : the screenshots are on corresponding question folder.

Q1

1. Problem Statement

The objective is to design a hardware circuit using Verilog HDL that determines if a binary number is divisible by 3. The specific constraints for this design are:

- **Input Format: Serial bit stream, starting from the Least Significant Bit (LSB).**
- **Architecture: Finite State Machine (FSM) in Mealy configuration.**
- **Output: A signal `rem` which is 0 if divisible and 1 if not divisible.**
- **Constraints: No high-level modulo (`%`) operators; the design must be synthesizable gate-level logic.**

2. Mathematical Theory

To check divisibility by 3 processing LSB first, we rely on the properties of modular arithmetic regarding powers of 2:

N=i=0Σnbi·2i

We need to calculate $N \pmod 3$. Observing the powers of 2 modulo 3:

- **$2^0 \equiv 1 \pmod 3$**
- **$2^1 \equiv 2 \pmod 3$**
- **$2^2 = 4 \equiv 1 \pmod 3$**
- **$2^3 = 8 \equiv 2 \pmod 3$**

Observation: The "weight" of the incoming bit alternates between 1 (for even positions: bit 0, 2, 4...) and 2 (for odd positions: bit 1, 3, 5...).

Therefore, to calculate the running remainder, the FSM must track two variables:

1. **Current Remainder: The accumulated remainder so far ($0, 1, \text{or } 2$).**
2. **Current Weight: The modulo-3 weight of the bit currently being processed ($1 \text{ or } 2$).**

3. FSM Design (Mealy Machine)

3.1 State Definition

Since we track 3 possible remainders and 2 possible weights, the FSM requires $3 \times 2 = 6$ states.

State	Description	Current Remainder	Bit Weight Expected
S0	Initial State	0	1
S1	Rem 1, Weight 1	1	1
S2	Rem 2, Weight 1	2	1
S3	Rem 0, Weight 2	0	2
S4	Rem 1, Weight 2	1	2
S5	Rem 2, Weight 2	2	2

3.2 Transition Logic

The next remainder is calculated as:

Next_Rem=(Current_Rem+(Input_Bit×Current_Weight))(mod3)

- **If Current Weight is 1, the Next Weight becomes 2.**
- **If Current Weight is 2, the Next Weight becomes 1.**

3.3 State Transition Table

Current State	Input (0)	Next State (0)	Output (0)	Input (1)	Next State (1)	Output (1)
S0 (R0, W1)	0	S3	0 (Divisible)	1	S4	1
S1 (R1, W1)	0	S4	1	1	S5	1
S2 (R2, W1)	0	S5	1	1	S3	0 (Divisible)
S3 (R0, W2)	0	S0	0 (Divisible)	1	S2	1
S4 (R1, W2)	0	S1	1	1	S0	0 (Divisible)
S5 (R2, W2)	0	S2	1	1	S1	1

(Note: Output is 0 if the resulting New Remainder is 0).

---

4. Implementation Details

The design was implemented in Verilog for Xilinx Vivado.

- **Module Name: `div3_fsm_lsb`**
- **Encoding: 3-bit binary encoding was used for the states.**

- **Reset Logic:** Asynchronous active-high reset sets the FSM to State S0.
- **Mealy Output:** The output logic `rem` is purely combinational, derived from `current_state` and the immediate `bit_in`. This ensures the output is valid as soon as the input bit is stable, without waiting for the next clock edge.

---

5. Simulation & Verification

5.1 Testbench Strategy

A custom testbench (`tb_div3_fsm_lsb`) was created to automate verification.

- **Input Vectoring:** A task `check_number` accepts an integer, converts it to binary, and feeds it into the DUT (Device Under Test) LSB-first.
- **Timing Control:**
  - Since this is a Mealy machine, the output changes *immediately* when the input changes.
  - Crucial Timing Detail: The testbench must sample the `rem` output after the input bit is stable but before the rising edge of the clock (which moves the FSM to the next state). Sampling after the clock edge would result in reading the logic for the *next* bit calculation, causing errors.

5.2 Test Vectors and Results

The following numbers were simulated to cover all state transitions and edge cases:

Number	Binary (LSB $\to$ MSB)	Mathematical Expected	Simulation Result	Pass/Fail
3	`11`	Divisible (Rem 0)	Rem 0	PASS
6	`011`	Divisible (Rem 0)	Rem 0	PASS
5	`101`	Not Divisible (Rem 2)	Rem 1	PASS

9	`1001`	Divisible (Rem 0)	Rem 0	PASS
0	`0`	Divisible (Rem 0)	Rem 0	PASS
14	`0111`	Not Divisible (Rem 2)	Rem 1	PASS
43690	`0101...`	Not Divisible (Rem 1)	Rem 1	PASS

5.3 Simulation Log Snapshot

Testing Number: 3 (Binary LSB->MSB: 11)
-> Result: DIVISIBLE (rem=0) | Status: PASS

Testing Number: 6 (Binary LSB->MSB: 011)
-> Result: DIVISIBLE (rem=0) | Status: PASS

Testing Number: 5 (Binary LSB->MSB: 101)
-> Result: Remainder (rem=1) | Status: PASS

6. Conclusion

The 6-state Mealy FSM successfully detects divisibility by 3 for serial LSB-first input streams. The initial simulation failures were identified as timing synchronization issues in the testbench (sampling Post-Clock vs Pre-Clock). Once the testbench was corrected to adhere to Mealy machine timing requirements, the design passed all test cases, proving the logic is correct and synthesizable.

Q2

Part (a): Adjustable Clock Enable Generator

Design Concept:
To slow down a module without using a gated clock (which causes timing glitches) or a DCM (which is complex hardware), we generate a Clock Enable (CE) pulse.

- **If we want the circuit to run at full speed (1x), we keep CE high constantly.**
- **If we want the circuit to run N times slower, we assert CE for exactly one clock cycle every N cycles.**
- **The input speed_div is an 8-bit number (0 to 255).**
    - **If input is 0, we divide by 1 (Full Speed).**
    - **If input is 1, we divide by 2.**
    - **If input is 255, we divide by 256.**
    - **Formula: $Period = \text{Input} + 1$.**

Part (b): Adjustable Speed Counter

Design Concept:
This module demonstrates how to use the ce_out signal.

1. **We instantiate the clk_enable_gen module.**
2. **We create a standard counter logic inside adjustable_counter.**
3. **Crucially, the counter logic is clocked by the system clk, but the state only updates if (ce_wire == 1). This effectively slows down the operation without physically changing the clock frequency.**

Explanation of Operation

1. **Why not Gated Clock?**
    - ***Gated Clock approach:*** **assign slow_clk = clk & enable. This is bad practice in FPGA design because it introduces "skew" (timing delays) and "glitches" (narrow pulses that confuse flip-flops).**
    - ***Clock Enable approach (Used here):*** **The global clock clk reaches every flip-flop simultaneously. The ce signal simply tells the flip-flops "don't change your value this cycle" (hold) or "update your value now". This is fully synchronous and safe.**
2. **Speed Calculation:**

- **Case 1: speed_setting = 0**
  - **The counter in Part A continuously reloads 0.**
  - **ce_out stays HIGH every cycle.**
  - **The counter in Part B updates on every rising edge (Full Speed).**
- **Case 2: speed_setting = 1**
  - **Part A counter goes: 1 -> 0 (Pulse) -> 1 -> 0 (Pulse).**
  - **ce_out is High every *other* cycle.**
  - **Speed is 1/2 of maximum.**
- **Case 3: speed_setting = 255**
  - **Part A counter counts down from 255 to 0.**
  - **ce_out pulses once every 256 cycles.**
  - **Speed is 1/256 of maximum.**

Q3:

To design a FIFO using two stacks, we utilize the classic algorithm:

1. **Enqueue operations push data onto Stack A (Input Stack).**
2. **Dequeue operations pop data from Stack B (Output Stack).**
3. **When Stack B is empty and a read is requested, we pop all elements from Stack A one by one and push them onto Stack B. This reverses the order twice (LIFO + LIFO = FIFO).**

1. Stack Module Design

First, we define a parameterized Stack module. It uses an internal register array (inferring distributed RAM or registers) and a stack pointer.

2. FIFO Module (Two-Stack Implementation)

This module instantiates two stacks and contains a State Machine to handle the data transfer.

Key FSM logic:

- **IDLE: Handles normal writes to Stack A and reads from Stack B.**

- **MOVE_POP: If a read is requested but B is empty (and A is not), we enter this state to pop from A.**
- **MOVE_PUSH: We push the data retrieved from A into B. We repeat the loop until A is empty.**

Design Strategy

The design strictly adheres to the constraint of using two instantiated Stack modules. No circular pointer arithmetic (traditional FIFO) was used.

- **Stack A (Input): Handles all incoming data.**
- **Stack B (Output): Handles all outgoing data.**
- **FIFO Order Preservation:**
    - **Pushing 1, 2, 3 to A results in A: [1, 2, 3] (Top is 3).**
    - **Popping A and Pushing B reverses this:**
        - **Pop 3, Push B -> B: [3]**
        - **Pop 2, Push B -> B: [3, 2]**
        - **Pop 1, Push B -> B: [3, 2, 1] (Top is 1).**
    - **Popping B now yields 1, then 2, then 3. This is FIFO order.**

State Machine Operation

The busy signal is crucial here. Since the transfer from A to B takes multiple clock cycles ($2 \times N$ cycles for N items), the FIFO cannot service a read immediately if Stack B is empty.

1. **Read Request (Empty B): The FSM detects rd_en + b_empty + !a_empty.**
2. **Transfer Phase: The FSM asserts busy. It cycles through POP_A and PUSH_B states until Stack A is drained.**
3. **Completion: The FSM returns to IDLE. Stack B is now populated. The user must hold or re-assert rd_en (depending on the external system design) to finally read the data from Stack B.**

Constraints Check

- **(a) FIFO Order: Achieved via the double-reversal property of two stacks.**

- **(b) Transfer on Read: The transfer is triggered specifically when a read is requested while the output stack is empty.**
- **(c) Synchronous: All updates happen on posedge clk.**
- **(d) No Gated Clocks: The clk signal is fed directly to all modules; enables are handled via logic (push/pop signals).**

1. Introduction

The objective of this project was to design a First-In-First-Out (FIFO) data buffer without utilizing the traditional circular buffer method or pre-existing IP cores. Instead, the design leverages two Last-In-First-Out (LIFO) stacks (StackA and StackB) to emulate FIFO behavior.

This architecture is a classic algorithmic problem where the order of data is reversed twice (Reverse $\to$ Reverse = Original Order) to achieve the FIFO property. The system was implemented in Verilog HDL, adhering to strict synchronous design practices constraints.

2. Theory of Operation

2.1 The Two-Stack Algorithm

A Stack is inherently LIFO. To create a FIFO, we divide the responsibility between two stacks:

1. **Input Stack (Stack A): Accepts all incoming write data.**
2. **Output Stack (Stack B): Provides all outgoing read data.**

The logic follows these rules:

- **Enqueue (Write): Always push data onto Stack A.**
- **Dequeue (Read):**
    - **Case 1: If Stack B is not empty, pop from Stack B. (Latency: 1 cycle).**
    - **Case 2: If Stack B is empty, we must transfer contents. We pop all elements from Stack A one by one and push them onto Stack B. Once the transfer is complete, we pop from Stack B. (Latency: $2 \times N$ cycles).**

2.2 Mathematical Validity

Let the input sequence be $1, 2, 3$.

1. **Push to A: A = $[1, 2, 3]$ (Top is 3).**
2. **Transfer A to B:**
    - **Pop 3 from A, Push to B $\to$ B = $[3]$**
    - **Pop 2 from A, Push to B $\to$ B = $[3, 2]$**
    - **Pop 1 from A, Push to B $\to$ B = $[3, 2, 1]$ (Top is 1).**
3. **Pop from B: Returns 1, then 2, then 3.**
   Result: FIFO order is preserved.

3. Hardware Design

3.1 Component: The Stack Module

A parameterized Stack module was created to serve as the building block.

- **Storage: Inferable distributed RAM or Register Array.**
- **Pointers: A stack pointer (ptr) tracks the top of the stack.**
- **Flags: full (ptr = DEPTH) and empty (ptr = 0).**
- **Operation: Synchronous PUSH and POP operations.**

3.2 Component: The FIFO Controller (FSM)

The core logic is managed by a Finite State Machine (FSM) that oversees the transfer process.

State Definitions:

1. **IDLE (00): The default state. It handles normal Writes to A and normal Reads from B. If a Read is requested but B is empty and A is not, it transitions to the transfer sequence.**
2. **MOVE_POP (01): Reads the top value from Stack A.**
3. **MOVE_PUSH (10): Writes the value retrieved from Stack A into Stack B. Loops back to MOVE_POP until Stack A is empty.**

Signal Description:

- **clk, rst: Global system clock and active-high reset.**
- **wr_en: Write Enable (User command).**
- **rd_en: Read Enable (User command).**
- **busy: Critical Status Output. Since the transfer from A to B is not instantaneous, this flag informs the external system that the FIFO is internally reorganizing and cannot service a new Read request immediately.**

4. Implementation Constraints & Compliance

Constraint	Implementation Strategy	Status
(a) FIFO Order	**Achieved via the dual-stack reversal algorithm (A $\to$ B).**	**Met**
(b) Auto-Transfer	**FSM detects rd_en && b_empty and triggers the MOVE states.**	**Met**
(c) Synchronous	**All sequential logic is triggered on posedge clk.**	**Met**

(d) No Gated Clocks	**The global clock drives all flip-flops. Logic enables (if statements) are used instead of gating the clock line.**	**Met**

Q3

1. Input Condition Logic (Solving Hardware Issues)

The raw mechanical button signal btn_async is unreliable. We implemented a three-stage conditioning pipeline to solve the reported issues:

- **Issue (a): Asynchronous Input**
  - **Solution: A 2-Stage Synchronizer (btn_meta, btn_sync) is used.**
  - **Mechanism: The asynchronous signal is clocked through two flip-flops back-to-back. This greatly reduces the probability of metastability propagating to the rest of the circuit, ensuring the FSM sees a stable logic level aligned with the clock.**
- **Issue (b): Multiple State Transitions (Bouncing)**
  - **Solution: A Debouncer logic is added.**
  - **Mechanism: A counter increments only when the input is stable. If the input flips (bounces), the counter resets. The internal signal btn_debounced only changes state after the input has remained stable for DEBOUNCE_LIMIT cycles.**
- **Issue (c): Unexpected States / Unclean Events**
  - **Solution: An Edge Detector (Pulse Generator).**
  - **Mechanism: The FSM does not check if the button is "High"; it checks if the button has "Just gone High" (btn_event). This is generated by btn_debounced & ~btn_prev. This ensures that holding the button down creates only one event, preventing the FSM from looping rapidly or detecting false "second presses" in the Active state.**

2. FSM Design (Functional Requirements)

The FSM is implemented using a standard Moore/Mealy hybrid architecture (Sequential State Logic + Combinational Next State Logic).

- **Reset (S_RESET): Acts as a safe entry point. It unconditionally transitions to IDLE on the next clock edge, ensuring the system starts in a known state.**
- **Idle (S_IDLE): The system waits here with outputs off. It only exits on btn_event.**
- **Active (S_ACTIVE):**
  - **A synchronous counter (timer_cnt) counts from 0 to 9 (10 cycles).**
  - **Priority Logic: The requirement "If user generates another button event... transition to Error" is placed *before* the timeout check. This ensures safety interrupts are handled immediately.**
- **Error (S_ERROR): This is a "trap" state. No logic leads out of this state; the only exit is the hardware rst signal.**

3. Constraints Checklist

- **(a) Asynchronous Input: Handled via synchronizer chain.**
- **(b) Synchronous Transitions: All FSM updates occur on posedge clk.**
- **(c) Avoid Combinational Loops: The next_state logic is purely combinational based on current registers, with no feedback loops inside the combinational block. Outputs are registered.**

1. Executive Summary

This report documents the design and implementation of a Finite State Machine (FSM) based controller for a smart appliance. The primary challenge was to interface a synchronous digital system with an asynchronous, noisy mechanical push-button.

The design implements a multi-stage Input Conditioning Pipeline (Synchronizer, Debouncer, and Edge Detector) to filter the input before it reaches the main control logic. The system was implemented in Verilog HDL and verified using a behavioral testbench to ensure immunity to metastability, switch bouncing, and logic hazards.

2. Problem Statement & Analysis

During initial hardware testing, three critical failures were observed:

1. **Metastability: The asynchronous button signal violated setup/hold times of the internal flip-flops.**
2. **Switch Bouncing: Physical contact vibrations caused the FSM to interpret a single press as multiple rapid events.**
3. **Level Sensitivity: Holding the button down caused the FSM to cycle through states unintentionally (e.g., entering Active and immediately triggering Error).**

3. System Architecture

To address these issues, the design is split into two distinct modules within the top-level hierarchy:

1. **Input Conditioning Path: Cleans the raw signal.**
2. **Main Control FSM: Executes the system logic based on the clean signal.**

3.1 Detailed Data Flow

Raw InputSyncStable SignalDebounceClean LevelEdge DetectSingle PulseFSMOutputs

---

4. Detailed Design Description

4.1 Stage 1: Synchronization (Metastability Protection)

- **Problem: An asynchronous signal entering a synchronous system can cause a flip-flop to enter a metastable state (neither 0 nor 1), potentially crashing the system.**
- **Solution: A 2-stage Shift Register is used.**
    - **The first flip-flop samples the asynchronous input. It may go metastable.**
    - **The second flip-flop samples the output of the first. By the next clock cycle, the probability of the first flip-flop remaining metastable is statistically negligible.**

- **Verilog:** `always @(posedge clk) {btn_sync, btn_meta} <= {btn_meta, btn_async};`

4.2 Stage 2: Digital Debouncing (Noise Filtering)

- **Problem:** Mechanical switches bounce for 5ms–20ms, creating a train of `1-0-1-0` pulses.
- **Solution:** A Counter-based Debouncer.
    - The logic compares the incoming synchronized bit with the currently stored stable state.
    - If they differ, a counter increments.
    - If they match, the counter resets.
    - The state is updated only if the counter reaches the `DEBOUNCE_LIMIT`. This acts as a low-pass filter.

4.3 Stage 3: Edge Detection (Pulse Generation)

- **Problem:** The FSM interprets a "High" level as a command. If the user holds the button for 500ms, the FSM sees this as "active" for millions of clock cycles. This caused the transition from Active $\to$ Error to happen instantly.
- **Solution:** A Rising-Edge Detector.
    - **Logic:** `is_rising_edge = current_val & ~previous_val;`
    - This converts a button press of *any duration* into a control signal that lasts exactly one clock cycle.

4.4 Stage 4: Finite State Machine (Controller)

The FSM is a 4-state Moore/Mealy hybrid.

State Name	Encoding	Behavior	Transition Logic
RESET	`00`	Outputs Low.	Automatically moves to IDLE on next clock.

IDLE	01	Outputs Low. Waits for user.	If btn_event detected $\to$ ACTIVE.
ACTIVE	10	appliance_on = 1. Timer counts 0-9.	1. If btn_event $\to$ ERROR (Safety Priority).2. Else if timer == 9 $\to$ IDLE.
ERROR	11	fault_status = 1. System Locked.	Remains in ERROR until Hardware Reset.

---

5. Design Considerations & Safety

5.1 Synchronous Design

- No Gated Clocks: The system clock clk drives every flip-flop. Enable signals are used for logic control.
- Reset Strategy: An asynchronous system reset rst is used to force the system into the RESET state immediately, clearing all counters and error flags.

5.2 Priority Logic in ACTIVE State

A critical design requirement was: *"If user generates another button event during this state, the system transitions immediately to Error."*
In the Verilog code, the check for the button event is placed before the check for the timer expiration.

```
if (btn_event)      next_state = S_ERROR;  // Higher Priority
else if (timer_done) next_state = S_IDLE;   // Lower Priority
```

This ensures that if a user panic-presses the button at the exact moment the timer expires, the system safely defaults to the Error state rather than shutting down.